questionario_06_RASTERIZACAO

Parte 1 — Múltipla escolha

Questão 1 — Bresenham, cálculo da variável de decisão

Para o segmento de (0,0) a (8,3), com $\Delta x = 8$ e $\Delta y = 3$, use $p0 = 2\Delta y - \Delta x$; se $pk < 0$, $pk{+}1 = pk + 2\Delta y$; se $pk \geqslant 0$, $pk{+}1 = pk + 2\Delta y - 2\Delta x$. Qual é a sequência de pk para k = 0, …, 7?

- a) −5, −2, 1, −4, −1, 2, −3, 0
- b) −2, 4, −6, 0, −10, −4, 2, −8
- c) −2, −12, −22, −32, −42, −52, −62, −72
- d) 13, 3, −7, −1, 5, −5, 1, −9
- e) −2, 4, 2, 0, −2, 4, 2, 0

Questão 2 — Interpolação com baricêntricas

Triângulo A(1,1), B(9,3), C(3,7), com $\text{área}_2 = 44$ e profundidades zA = 0,2; zB = 0,8; zC = 0,5. Para o ponto R(5,3), as funções de aresta valem $w_0 = E(B,C,R) = 16$, $w_1 = E(C,A,R) = 20$ e $w_2 = E(A,B,R) = 8$. Qual é a profundidade interpolada em R?

- a) ≈ 0,555
- b) ≈ 0,582
- c) 0,500
- d) ≈ 0,527
- e) ≈ 0,459

Questão 3 — Funções de aresta nas GPUs

Por que as funções de aresta, e não o scanline, se tornaram a abordagem padrão para rasterizar triângulos nas GPUs atuais?

- a) Porque funcionam diretamente com polígonos côncavos de qualquer número de lados, sem triangulação.
- b) Porque dispensam a caixa envolvente: testam diretamente só os pixels dentro do triângulo.
- c) Porque percorrem uma linha por vez, reaproveitando as interseções da linha anterior.
- d) Porque cada pixel da caixa envolvente é testado de forma independente, com as mesmas três contas, o que se paraleliza muito bem; além disso, as baricêntricas saem do mesmo cálculo.
- e) Porque eliminam a necessidade do Z-buffer, já que determinam a visibilidade entre triângulos.

Questão 4 — Z-buffer

Um pixel tem zbuf = 1,0 (valor inicial). Três fragmentos chegam a ele, nesta ordem: vermelho com z = 0,70; azul com z = 0,40; verde com z = 0,55. Pelo teste "passa se z < zbuf", qual é a cor final e o valor final de zbuf?

- a) Azul, com zbuf = 0,40
- b) Verde, com zbuf = 0,55
- c) Vermelho, com zbuf = 0,70
- d) Uma mistura das três cores, ponderada pelas profundidades.
- e) Azul, com zbuf = 0,55

Questão 5 — Cobertura e antialiasing

Um pixel tem exatamente 37,5% da sua área dentro de um triângulo. Segundo o exemplo da aula, qual afirmação está correta?

- a) Com apenas uma amostra no centro do pixel, a cobertura estimada seria de 37,5%.
- b) Com amostragem 4×4, 6 das 16 amostras caem dentro (37,5%), coincidindo com a cobertura exata; a cor final é cobertura · cor do triângulo + (1 − cobertura) · cor de fundo.
- c) O MSAA 4× chega ao mesmo resultado do SSAA executando o fragment shader 4 vezes por pixel de borda.
- d) Com amostragem 2×2, a cobertura estimada é de 37,5%, pois 2 de 4 amostras caem dentro.
- e) O FXAA calcula a cobertura exata do pixel contando amostras geométricas adicionais.

Parte 2 — Verdadeiro ou falso

Questão 6. Na grade de pixels, o pixel (i, j) tem centro em (i + 0,5; j + 0,5). ( ) V ( ) F

Questão 7. A rasterização recebe os vértices ainda em coordenadas de recorte homogêneas, antes da divisão por w e da matriz de viewport. ( ) V ( ) F

Questão 8. O algoritmo DDA avança x de 1 em 1 e soma a inclinação m a y a cada passo, plotando (x, round(y)); por isso depende de aritmética em ponto flutuante. ( ) V ( ) F

Questão 9. No caso base do Bresenham ($0 \leqslant m \leqslant 1$, $x_0 < x_1$), $p_0 = 2\Delta y - \Delta x$ e, quando $pk < 0$, o próximo pixel é (x+1, y). ( ) V ( ) F

Questão 10. Para segmentos com m > 1, o Bresenham do caso base pode ser aplicado sem nenhuma adaptação. ( ) V ( ) F

Questão 11. Para o segmento (0,0) → (8,3), o Bresenham acende pixels diferentes dos do DDA, porque usa inteiros em vez de ponto flutuante. ( ) V ( ) F

Questão 12. No scanline o preenchimento de uma linha depende do estado da linha anterior (arestas ativas), enquanto nas funções de aresta cada pixel pode ser testado de forma independente. ( ) V ( ) F

Questão 13. O sinal de E(a, b, p) indica de que lado da aresta a→b o ponto p está, e E = 0 quando p está sobre a reta que contém a aresta. ( ) V ( ) F

Questão 14. Para o triângulo A(1,1), B(9,3), C(3,7), $área_2$ = E(A,B,C) = 44, e as baricêntricas de um ponto interno, calculadas como wi/$área_2$, somam 1. ( ) V ( ) F

Questão 15. Se um dos pesos baricêntricos de P é negativo, P ainda pode estar dentro do triângulo, desde que os outros dois sejam positivos. ( ) V ( ) F

Questão 16. Um pixel exatamente sobre uma aresta compartilhada por dois triângulos deve ser desenhado pelos dois, para que não surjam buracos na malha. ( ) V ( ) F

Questão 17. Como a interpolação linear em coordenadas de tela é exata para qualquer atributo, não é necessário corrigir a perspectiva ao interpolar coordenadas de textura. ( ) V ( ) F

Questão 18. No Z-buffer inicializado com 1,0, se zbuf(4,3) = 0,459 e chega um fragmento com z = 0,62 nesse pixel, o fragmento é descartado e cor e profundidade permanecem inalteradas. ( ) V ( ) F

Questão 19. O MSAA executa o fragment shader uma vez por amostra, por isso também suaviza o aliasing de texturas e de shaders. ( ) V ( ) F

Questão 20. O TAA desloca (jitter) a amostragem a cada quadro e acumula o histórico usando vetores de movimento, o que pode gerar ghosting. ( ) V ( ) F

Gabarito

Questão	Resposta	Questão	Resposta
1	b	11	F
2	d	12	V
3	d	13	V
4	a	14	V
5	b	15	F
6	V	16	F

Questão	Resposta	Questão	Resposta
7	F	17	F
8	V	18	V
9	V	19	F
10	F	20	V

Comentários

1 — b. $2\Delta y = 6$ e $2\Delta y - 2\Delta x = -10$. p0 = 6 − 8 = −2; p1 = −2 + 6 = 4; p2 = 4 − 10 = −6; p3 = 0; p4 = −10; p5 = −4; p6 = 2; p7 = −8.

2 — d. Cada peso multiplica a profundidade do vértice oposto à aresta: $z = (16 \cdot 0{,}2 + 20 \cdot 0{,}8 + 8 \cdot 0{,}5)/44 = 23{,}2/44 \approx 0{,}527$.

3 — d. Cada pixel é testado de forma independente com as mesmas três funções lineares, o que se encaixa no paralelismo SIMD das GPUs; os mesmos valores normalizados já são as baricêntricas usadas na interpolação.

4 — a. Vermelho passa (0,70 < 1,0); azul passa (0,40 < 0,70); verde é descartado (0,55 não é menor que 0,40). Final: azul, zbuf = 0,40.

5 — b. 6/16 = 37,5%, igual à cobertura exata, e a cor final mistura triângulo e fundo pela cobertura. Uma amostra só dá 0% ou 100%; 2/4 seria 50%; o MSAA executa o shader uma vez por pixel; o FXAA é pós-processamento sem amostras geométricas.

6 — V. O pixel (i, j) ocupa $[i, i+1] \times [j, j+1]$; seu centro é $(i + 0{,}5;\ j + 0{,}5)$, ponto onde se avaliam as funções de aresta.

7 — F. Antes da rasterização ocorrem o recorte, a divisão perspectiva por w (NDC) e a transformação de viewport; a rasterização trabalha em coordenadas de tela.

8 — V. O DDA acumula $y = y + m$ e arredonda a cada passo, exigindo ponto flutuante e acumulando erro.

9 — V. Se $p_k < 0$, o ponto médio está acima da reta, então o pixel escolhido é $(x+1, y)$ e $p_{k+1} = p_k + 2\Delta y$.

10 — F. Para $m > 1$, y cresceria mais de 1 por passo e haveria buracos; trocam-se os papéis de x e y (avança-se y), usando simetrias para os demais octantes.

11 — F. Os dois acendem os mesmos pixels: (0,0), (1,0), (2,1), (3,1), (4,2), (5,2), (6,2), (7,3), (8,3). A diferença é o custo e a precisão do cálculo, não o resultado.

12 — V. O scanline atualiza incrementalmente a tabela de arestas ativas linha a linha; as funções de aresta avaliam cada pixel isoladamente.

13 — V. E(a, b, p) é o produto vetorial 2D entre a→b e a→p; o sinal indica o lado, e E = 0 indica p colinear com a aresta.

14 — V. E(A,B,C) = (9 − 1)(7 − 1) − (3 − 1)(3 − 1) = 48 − 4 = 44. Para um ponto interno, $w_0 + w_1 + w_2 = \text{área}_2$ (no exemplo de R: 16 + 20 + 8 = 44), logo as baricêntricas somam 1.

15 — F. P está dentro somente se os três pesos forem ⩾ 0; um peso negativo coloca P do lado de fora da aresta correspondente.

16 — F. O pixel deve ser desenhado por exatamente um triângulo, usando uma regra de desempate como a topo-esquerda; desenhar pelos dois causa dupla escrita e erros de blending.

17 — F. A projeção perspectiva não preserva linearidade em tela; interpolam-se u/w, v/w e 1/w e depois divide-se. A profundidade z′ após a divisão por w é linear em tela.

18 — V. 0,62 < 0,459 é falso, então o fragmento é descartado e o pixel mantém cor e zbuf = 0,459.

19 — F. O MSAA executa o shader uma vez por pixel e só avalia cobertura e profundidade por amostra; suaviza apenas bordas geométricas. Quem executa o shader por amostra é o SSAA.

20 — V. O TAA aplica jitter subpixel a cada quadro e combina com o histórico reprojetado por vetores de movimento; falhas na reprojeção deixam rastros (ghosting).